CARLOS EXPEDITO RODRIGUES ARAGÃO

POS TECH



IA PARA DEVS

PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 06

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....12
O QUE VOCÊ VIU NESTA AULA? 21
REFERÊNCIAS.....22

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

O QUE VEM POR AÍ?

Olá! Nesta última aula você vai recriar o projeto da aula anterior, aprendendo a utilizar a biblioteca SpaCy no lugar do NLTK e criar um modelo Word2Vec do zero sem uma base pré-treinada, assim você vai conseguir criar um classificador com o seu próprio modelo.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

HANDS ON

Nesta aula, implementaremos a criação de um modelo Word2Vec. Anteriormente, fizemos o pré-processamento dos dados utilizando a biblioteca SpaCy e fizemos nosso classificador baseado no nosso modelo criado.

Primeiro vamos importar o Pandas e trazer nossos dados.

```
import pandas as pd
artigo_treino = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Word2Vec/treino.csv")
artigo_teste = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Word2Vec/teste.csv")
```

Agora vamos baixar os dados do SpaCy.

```
!python -m SpaCy download pt_core_news_sm
```

Vamos importar o SpaCy e fazer o load dos dados baixados.

```
import SpaCy
nlp = SpaCy.load("pt_core_news_sm")
```

Vamos criar a variável textos_para_tratamento.

```
textos_para_tratamento = (titulos.lower() for titulos in artigo_treino.title)
```

E o método trata_textos.

```
def trata_textos(doc):
  tokens_validos = []
```

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

```
  for token in doc:
    e_valido = not token.is_stop and token.is_alpha
    if e_valido:
      tokens_validos.append(token.text)



 if len(tokens_validos) > 2:
    return " ".join(tokens_validos)
```

Agora vamos tratar os textos.

```
textos_tratados = [trata_textos(doc) for doc in nlp.pipe(textos_para_tratamento,
                                                         batch_size = 1000,
                                                         n_process = -1)]
```

E criar o data frame na variável títulos_tratados

```
titulos_tratados = pd.DataFrame({"titulo": textos_tratados})
```

Para criar o modelo, vamos importar o gensim e instanciar o modelo com os hiperparâmetros.

```
from gensim.models import Word2Vec


w2v_modelo = Word2Vec(sg = 0,
                      window = 2,
                      vector_size = 300,
                      min_count = 5,
                      alpha = 0.03,
```

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

```
                min_alpha = 0.007)
```

Vamos retirar dados nulos e duplicados.

```
titulos_tratados = titulos_tratados.dropna().drop_duplicates()
```

Criando uma lista de uma lista para passar para o modelo:

```
lista_lista_tokens = [titulo.split(" ") for titulo in titulos_tratados.titulo]
```

Pedir ao modelo para criar o vocabulário:

```
w2v_modelo.build_vocab(lista_lista_tokens)
```

Criando a classe callback para ver o resultado do cálculo do erro do modelo:

```
from gensim.models.callbacks import CallbackAny2Vec
from gensim.models import Word2Vec


# iniciando a chamada callback
class callback(CallbackAny2Vec):
  def __init__(self):
    self.epoch = 0


  def on_epoch_end(self, model):
    loss = model.get_latest_training_loss()
    if self.epoch == 0:
        print('Loss após a época {}: {}'.format(self.epoch, loss))
    else:
```

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

```
        print('Loss após a época {}: {}'.format(self.epoch, loss-self.loss_previous_step))
    self.epoch += 1
    self.loss_previous_step = loss
```

Treinar o modelo CBOW.

```
w2v_modelo.train(lista_lista_tokens,
                 total_examples = w2v_modelo.corpus_count,
                 epochs = 30,
                 compute_loss = True,
                 callbacks=[callback()])
```

Treinar o modelo Skip Gram

```
w2v_modelo_sg = Word2Vec(sg = 1,
                     window = 5,
                     vector_size = 300,
                     min_count = 5,
                     alpha = 0.03,
                     min_alpha = 0.007)

w2v_modelo_sg.build_vocab(lista_lista_tokens)


w2v_modelo_sg.train(lista_lista_tokens,
                 total_examples = w2v_modelo_sg.corpus_count,
                 epochs = 30,
                 compute_loss = True,
```

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

```
            callbacks=[callback()])
```

Podemos guardar nosso modelo em um arquivo txt no drive.

```
w2v_modelo.wv.save_word2vec_format("/content/drive/MyDrive/Colab Notebooks/Word2Vec/modelo_cbow.txt", binary=False)

w2v_modelo_sg.wv.save_word2vec_format("/content/drive/MyDrive/Colab Notebooks/Word2Vec/modelo_sg.txt", binary=False)
```

Iniciando o classificador.

```
import SpaCy

nlp = SpaCy.load("pt_core_news_sm", disable=["parser", "ner", "tagger", "textcat"])


def tokenizador(texto):
  tokens_validos = []
  doc = nlp(texto)


  for token in doc:
    e_valido = not token.is_stop and token.is_alpha
    if e_valido:
      tokens_validos.append(token.text.lower())


  return tokens_validos


texto = "Adoro a 342342 #$@#@#$ cidade de caldas novas!"

tokenizador(texto)
```

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Criando métodos necessários para classificação.

```
import numpy as np


def combinacao_de_vetores_por_soma(palavras, modelo):
  vetor_resultante = np.zeros(300)
  for pn in palavras:
    try:
      vetor_resultante =+ modelo.wv.get_vector(pn)
    except KeyError:
      pass


  return vetor_resultante
```

```
def matriz_vetores(textos, modelo):
  x = len(textos)
  y = 300
  matriz = np.zeros((x,y))


  for i in range(x):
    palavras = tokenizador(textos.iloc[i])
    matriz[i] = combinacao_de_vetores_por_soma(palavras, modelo)


  return matriz
```

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Criando matrizes de vetores de treino e teste CBOW e Skip Gram.

```
matriz_vetores_treino_cbow = matriz_vetores(artigo_treino.title, w2v_modelo)
matriz_vetores_teste_cbow = matriz_vetores(artigo_teste.title, w2v_modelo)
print(matriz_vetores_treino_cbow.shape)
print(matriz_vetores_teste_cbow.shape)

matriz_vetores_treino_sg = matriz_vetores(artigo_treino.title, w2v_modelo_sg)
matriz_vetores_teste_sg = matriz_vetores(artigo_teste.title, w2v_modelo_sg)
print(matriz_vetores_treino_sg.shape)
print(matriz_vetores_teste_sg.shape)
```

Criando modelo de regressão logística.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

def classificador(modelo, x_treino, y_treino, x_teste, y_teste):
  RL = LogisticRegression(max_iter = 800)
  RL.fit(x_treino, y_treino)
  categorias = RL.predict(x_teste)
  resultados = classification_report(y_teste, categorias)
  print(resultados)
  return RL
```

Comparando resultados.

```
RL_cbow = classificador(w2v_modelo,
                        matriz_vetores_treino_cbow,
```

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

```
                artigo_treino.category,
                matriz_vetores_teste_cbow,
                artigo_teste.category)
```

```
RL_cbow = classificador(w2v_modelo_sg,
                matriz_vetores_treino_sg,
                artigo_treino.category,
                matriz_vetores_teste_sg,
                artigo_teste.category)
```

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

SAIBA MAIS

O SpaCy é uma poderosa biblioteca de Processamento de Llinguagem Natural (PLN) em Python, desenvolvida pela Explosion AI. Ele se destaca por sua eficiência, desempenho rápido e design voltado para a produção, tornando-o uma escolha popular entre os desenvolvedores e pesquisadores na área de PLN.

Entre as principais características do SpaCy, destaca-se a tokenização eficiente, que permite dividir o texto em unidades significativas, como palavras e pontuações. Além disso, o SpaCy oferece uma análise morfológica detalhada, fornecendo informações sobre a estrutura gramatical das palavras, incluindo lemas, formas gramaticais e classes de palavras.

A análise sintática é outra funcionalidade robusta do SpaCy, permitindo a geração de árvores sintáticas para compreender a estrutura gramatical das sentenças. Isso inclui a identificação de sujeitos, objetos e relações entre palavras, proporcionando uma compreensão mais profunda do conteúdo linguístico.

O SpaCy também se destaca no reconhecimento de entidades nomeadas (NER), identificando automaticamente entidades como nomes de pessoas, organizações, locais, datas e outras informações específicas no texto. Esse recurso é fundamental para extrair informações relevantes em tarefas de análise de texto.

A biblioteca oferece suporte a modelos de aprendizado profundo pré-treinados para diversas tarefas, como análise de sentimento e classificação de texto. Além disso, o SpaCy é altamente extensível, permitindo que os usuários adicionem componentes personalizados para estender suas funcionalidades de acordo com as necessidades específicas.

Com modelos treinados para várias línguas, o SpaCy é uma escolha versátil para projetos multilíngues, facilitando a análise de texto em diferentes idiomas. Seu desempenho otimizado o torna adequado para tarefas em larga escala, sendo uma ferramenta valiosa para projetos que exigem processamento eficiente de grandes volumes de texto.

Em resumo, o SpaCy é uma ferramenta essencial para profissionais de PLN, oferecendo uma ampla gama de recursos e funcionalidades para análise e processamento avançado de texto. Sua facilidade de uso, documentação abrangente

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

e comunidade ativa tornam-no uma escolha preferida para aplicações de PLN em diversos contextos.

A análise morfológica no SpaCy é uma etapa fundamental no Processamento de Linguagem Natural (PLN) que se concentra na identificação e análise das características morfológicas das palavras em um texto. Essa análise fornece informações detalhadas sobre a estrutura gramatical das palavras, incluindo lemas, formas gramaticais e outras propriedades linguísticas essenciais.

Tokenização: antes de realizar a análise morfológica, o SpaCy inicia o processo com a tokenização, que envolve a divisão do texto em unidades significativas chamadas tokens. Esses tokens podem ser palavras, pontuações ou outras entidades gramaticais relevantes.

Análise Morfológica: após a tokenização, cada token passa por uma análise morfológica detalhada. Durante essa etapa, o SpaCy identifica várias características morfológicas, incluindo:

- **Lema (base da palavra**): representa a forma básica da palavra. Por exemplo, o lema de "correndo" é "correr".
- **Forma gramatical (POS - Part of Speech):** indica a classe gramatical da palavra, como substantivo, verbo, adjetivo, etc.
- **Informações de conjugação e flexão**: incluem informações sobre tempo, modo, número, gênero, entre outros, dependendo da classe gramatical da palavra.
- **Entidades gramaticais específicas**: identifica elementos específicos, como pronomes pessoais, pronomes possessivos, entre outros.

Exemplo Prático:

Vamos considerar a frase "Os pássaros estão voando alto" e verificar a análise morfológica de algumas palavras:

Para "pássaros", o lema seria "pássaro" e a forma gramatical indicaria um substantivo plural.

Para "estão", o lema seria "estar" e a forma gramatical indicaria um verbo no presente do indicativo.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Para "voando", o lema seria "voar" e a forma gramatical indicaria um verbo no gerúndio.

Aplicações Práticas: a análise morfológica é crucial em várias aplicações de PLN, incluindo tradução automática, análise de sentimentos, correção gramatical, entre outras. Compreender as características morfológicas das palavras ajuda a extrair significado e contexto do texto.

Personalização e Extensibilidade: o SpaCy permite que os usuários personalizem e estendam a análise morfológica adicionando regras personalizadas, dicionários ou treinando modelos específicos para casos de uso especializados.

Em resumo, a análise morfológica no SpaCy é uma etapa essencial para entender a estrutura gramatical de um texto. Ao fornecer informações detalhadas sobre as palavras, essa análise forma a base para tarefas mais avançadas de processamento de linguagem natural, contribuindo para a compreensão aprofundada e significativa do conteúdo textual.

A análise sintática é uma fase crucial no Processamento de Linguagem Natural (PLN) que se concentra na compreensão da estrutura gramatical e nas relações entre as palavras em uma sentença. No contexto do SpaCy, uma biblioteca avançada de PLN em Python, a análise sintática desempenha um papel fundamental na geração de árvores sintáticas que representam a estrutura gramatical de uma frase ou sentença.

Tokenização e Análise Morfológica: antes de iniciar a análise sintática, o texto é tokenizado para identificar unidades significativas, como palavras e pontuações. Em seguida, é realizada uma análise morfológica para extrair informações sobre lemas, formas gramaticais e outras características das palavras.

Dependência Sintática: o SpaCy utiliza um modelo de dependência sintática para criar árvores sintáticas que representam as relações gramaticais entre as palavras em uma sentença. Cada palavra na sentença é representada como um nó na árvore, e as relações entre elas são representadas por arestas.

Relações Gramaticais: a análise sintática identifica diversas relações gramaticais, incluindo sujeito, objeto direto, objeto indireto, complemento verbal, entre outras. Essas relações são cruciais para entender a função de cada palavra na estrutura da sentença.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Exemplo Prático:

Consideremos a frase "O gato está dormindo na janela" e analisemos a árvore sintática:

"gato" é o sujeito da frase.

"está" é o verbo de ligação.

"dormindo" é o predicado, indicando a ação.

"janela" é o objeto indireto, indicando onde o gato está dormindo.

Visualização da Árvore Sintática:

O SpaCy oferece recursos para visualizar a árvore sintática, o que é útil para compreender visualmente as relações gramaticais. Isso é especialmente valioso para análises manuais e depuração.

Aplicações Práticas: a análise sintática é fundamental em tarefas como extração de informações, tradução automática, geração de resumos automáticos, entre outras. Compreender a estrutura gramatical de uma sentença é essencial para interpretar adequadamente o significado do texto.

Personalização e Treinamento: o SpaCy permite a personalização e treinamento de modelos sintáticos para se adequar a domínios específicos ou melhorar o desempenho em casos de uso especializados.

Em resumo, a análise sintática no SpaCy é uma etapa crucial para compreender a estrutura gramatical e as relações entre as palavras em um texto. Ao gerar árvores sintáticas, o SpaCy capacita desenvolvedores e pesquisadores a extrair insights significativos do conteúdo textual, contribuindo para uma compreensão mais profunda e avançada do processamento de linguagem natural.

O Reconhecimento de Entidades Nomeadas (NER) é uma tarefa fundamental no Processamento de Linguagem Natural (PLN) que envolve a identificação e classificação de entidades específicas, como nomes de pessoas, organizações, locais, datas e outras informações importantes em um texto. No contexto do SpaCy, uma biblioteca avançada de PLN em Python, o NER é uma funcionalidade robusta que contribui significativamente para a extração de informações semânticas e o entendimento do conteúdo textual.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Identificação de Entidades: o SpaCy utiliza modelos pré-treinados para identificar automaticamente entidades nomeadas em um texto. Esses modelos são treinados em grandes conjuntos de dados para reconhecer padrões e características específicas associadas a diferentes tipos de entidades.

Tipos de Entidades:

O SpaCy classifica entidades em vários tipos, incluindo:

- Pessoas: nomes de indivíduos.
- Organizações: nomes de empresas, instituições, etc.
- Locais: nomes de lugares, como cidades, países, etc.
- Datas: referências temporais, como datas específicas.
- Quantidades: números que representam quantidades específicas.
- Outros tipos específicos: dependendo do modelo e do idioma, outras categorias podem incluir eventos, obras de arte, entre outros.

Contextualização e Ambiguidade: o SpaCy leva em consideração o contexto ao realizar o NER. Por exemplo, a entidade "Paris" pode ser uma cidade ou o nome de uma pessoa, mas o contexto na frase ajuda a determinar o significado correto.

Exemplo Prático:

Consideremos a frase "Bill Gates fundou a Microsoft em 1975" e a identificação de entidades pelo SpaCy:

"Bill Gates" seria identificado como uma pessoa.

"Microsoft" seria identificada como uma organização.

"1975" seria identificado como uma data.

Visualização de Entidades: o SpaCy fornece recursos para visualizar entidades diretamente no texto, destacando-as e indicando seus tipos, o que é útil para análises visuais e depuração.

Aplicações Práticas: o NER tem uma ampla variedade de aplicações práticas, incluindo:

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

- Recuperação de Informações: identificação de informações específicas em documentos.
- Sumarização Automática: auxilia na identificação de tópicos e eventos principais.
- Tradução Automática: ajuda na correspondência de entidades entre diferentes idiomas.
- Análise de Sentimento: contribui para a identificação de entidades relevantes em análises de opinião.

Personalização e Treinamento: o SpaCy permite a personalização do modelo NER para se adequar a domínios específicos ou para treinar modelos personalizados com dados anotados.

Em resumo, o Reconhecimento de Entidades Nomeadas no SpaCy é uma ferramenta poderosa para extrair informações relevantes e semânticas de textos, proporcionando uma base sólida para diversas aplicações em processamento de linguagem natural. Ao automatizar a identificação de entidades específicas, o SpaCy facilita a análise de grandes volumes de texto e contribui para uma compreensão mais rica e contextualizada do conteúdo textual.

Os embutimentos de palavras, também conhecidos como word embeddings, representam uma abordagem fundamental no campo do Processamento de Linguagem Natural (PLN) para representar palavras em um formato numérico contínuo. Essa representação vetorizada permite que algoritmos de aprendizado de máquina compreendam e processem a semântica e as relações entre palavras em um contexto textual. No SpaCy, uma biblioteca avançada de PLN em Python, os embutimentos de palavras são acessíveis por meio de modelos pré-treinados que capturam as nuances semânticas das palavras.

Representação Vetorial: em vez de tratar as palavras como unidades isoladas, os embutimentos de palavras as representam como vetores numéricos contínuos em um espaço n-dimensional. Cada dimensão no vetor captura uma característica semântica específica da palavra.

Distribuição Semântica: a ideia por trás dos embutimentos de palavras é que palavras semelhantes semanticamente devem ter representações vetoriais próximas

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

umas das outras. Essa propriedade permite que os modelos capturem relações semânticas e sinônimos.

Modelos Pré-treinados no SpaCy: o SpaCy oferece modelos pré-treinados que incorporam embutimentos de palavras para várias línguas. Esses modelos foram treinados em grandes conjuntos de dados e capturam as relações semânticas entre palavras em contextos diversos.

Similaridade Semântica: usando embutimentos de palavras, é possível calcular a similaridade semântica entre palavras ou frases. A proximidade no espaço vetorial indica a semelhança semântica. Por exemplo, os embutimentos de "gato" e "felino" seriam mais próximos do que os de "gato" e "computador".

Transferência de Aprendizado: os embutimentos de palavras treinados em grandes conjuntos de dados podem ser utilizados como ponto de partida em tarefas específicas de PLN, facilitando a transferência de aprendizado. Essa técnica é particularmente útil quando os conjuntos de dados de treinamento são limitados.

Personalização e Treinamento: o SpaCy permite que usuários personalizem ou treinem seus próprios modelos de embutimentos de palavras para se adequarem a contextos ou domínios específicos, agregando mais informações relevantes para tarefas específicas.

Aplicações Práticas: os embutimentos de palavras têm uma ampla gama de aplicações práticas, incluindo:

- Recuperação de Informações: identificação de documentos ou parágrafos relevantes.
- Classificação de Texto: auxílio na categorização de documentos.
- Tradução Automática: melhoria na correspondência semântica entre idiomas.
- Análise de Sentimento: captura de nuances semânticas nas opiniões expressas.

Visualização e Exploração: ferramentas no SpaCy permitem visualizar e explorar os embutimentos de palavras, oferecendo insights sobre as relações semânticas entre diferentes termos.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

Os embutimentos de palavras são uma peça-chave no avanço das capacidades de PLN, possibilitando uma compreensão mais rica e contextualizada do significado das palavras em textos. Ao representar palavras em um espaço vetorial, o SpaCy fornece uma ferramenta poderosa para análise e processamento avançado de linguagem natural em uma variedade de cenários e aplicações.

A Análise de Sentimento é uma área do processamento de linguagem natural (PLN) que se concentra na identificação e avaliação das emoções, opiniões e atitudes expressas em um texto. No SpaCy, uma biblioteca avançada de PLN em Python, a análise de sentimento pode ser realizada por meio de modelos treinados para classificar o tom geral de um texto em categorias como positivo, negativo ou neutro.

Classificação de Texto: a análise de sentimento envolve a classificação de um texto em categorias que representam a polaridade das opiniões expressas. Geralmente, essas categorias incluem positivo, negativo e neutro.

Modelos Pré-treinados: o SpaCy oferece modelos pré-treinados para análise de sentimento em vários idiomas. Esses modelos foram treinados em conjuntos de dados diversificados para reconhecer padrões linguísticos associados a diferentes tonalidades emocionais.

Treinamento Personalizado: os usuários têm a opção de treinar modelos de análise de sentimento personalizados para se adequarem a domínios específicos ou para melhorar o desempenho em cenários particulares.

Identificação de Opiniões: a análise de sentimento não se limita apenas a atribuir uma polaridade geral; ela também pode envolver a identificação de opiniões específicas sobre aspectos particulares de um produto, serviço ou tópico.

Desafios e Complexidades: a análise de sentimento enfrenta desafios, como a compreensão de ironias, sarcasmo e contextos ambíguos, que podem afetar a precisão dos resultados. Modelos mais avançados buscam lidar com essas complexidades.

Aplicações Práticas: a Análise de Sentimento é amplamente utilizada em diversas aplicações, incluindo:

- Monitoramento de Redes Sociais: avaliação das opiniões sobre produtos, marcas ou eventos.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

- Feedback do Cliente: análise de avaliações e comentários para entender a satisfação do cliente.
- Análise de Resenhas: classificação de resenhas de produtos, filmes, livros, etc.
- Gestão de Reputação Online: avaliação da reputação de empresas com base em opiniões online.

Limitações e Considerações Éticas: a análise de sentimento pode ser influenciada por vieses nos dados de treinamento, e resultados imprecisos podem surgir devido a variações culturais e linguísticas. Além disso, questões éticas relacionadas à privacidade devem ser consideradas ao lidar com textos que contenham informações pessoais.

Integração com Outras Tarefas de PLN: a Análise de Sentimento muitas vezes é combinada com outras tarefas de PLN, como reconhecimento de entidades nomeadas (NER) e análise de tópicos, para uma compreensão mais holística do conteúdo textual.

Em resumo, a Análise de Sentimento no SpaCy proporciona uma abordagem eficaz para extrair insights valiosos das opiniões expressas em textos. Ao automatizar a avaliação de tonalidades emocionais, o SpaCy oferece uma ferramenta valiosa para empresas e pesquisadores que desejam compreender as percepções do público em relação a produtos, serviços e tópicos específicos.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu os conceitos da biblioteca SpaCy e o quão poderosa ela pode ser no pré-processamento dos dados, também aprendeu a criar seu modelo Word2Vec com as técnicas CBOW e Skip Gram e recriou o classificador da última aula com seu modelo treinado e observou uma semelhança na acurácia das duas técnicas.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

REFERÊNCIAS

GENSIM. **Documentação Gensim**. [s.d]. Disponível em: <https://radimrehurek.com/gensim/models/keyedvectors.html>.Acesso em: 17 jan. 2024.

MIKOLOV, T. CHEN, K. CORRADO, G. DEAN, J. **Efficient Estimation of Word Representations in Vector Space**. 2013. Disponível em: <https://arxiv.org/pdf/1301.3781.pdf>. Acesso em: 17 jan. 2024.

NILC. **Repositório de Word Embeddings do NILC**. [s.d]. Disponível em: <http://nilc.icmc.usp.br/nilc/index.php/repositorio-de-word-embeddings-do-nilc>. Acesso em: 17 jan. 2024.

SPACY. **Documentação SpaCy**. [s.d]. Disponível em: <https://SpaCy.io/>.Acesso em: 17 jan. 2024.

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com

PALAVRAS-CHAVE

Word2Vec. SpaCy. CBOW. SkipGram.

FIAP

PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com



PDF exclusivo para André Pires de Oliveira Capela - rm376905
andr.capela@gmail.com